

```
In [4]: import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

import warnings
warnings.filterwarnings(action = 'ignore')

from IPython.display import Image
```

```
In [3]: Image('189.png')
```

Out[3]: **4-3 나이브 베이즈, K최근접 이웃**

0. 데이터셋 소개

SMS Spam dataset

[Spam](#): SMS 텍스트로 메시지와 Spam 여부를 나타낸 데이터셋입니다.

Name	Type	Description
target	binary	ham or spam
message	text	SMS message

텍스트를 입력으로 분류를 연습하기 위한 데이터셋입니다.

```
In [2]: df_spam = pd.read_csv('data/SMSSpamCollection.tsv', sep='\t')
df_spam.head()
```

Out[2]:

	target	message
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

In [3]: # 공백으로 문장을 나누어 단어의 수를 출력합니다.
df_spam['message'].str.split(' ').apply(len)

Out[3]:

0	20
1	6
2	28
3	11
4	13
	..
5567	30
5568	8
5569	10
5570	26
5571	6

Name: message, Length: 5572, dtype: int64

In [4]: Image('190.png')

Out[4]:

1. 나이브 베이즈(Naive Bayes)

- 베이즈 정리(Bayes' Theorem)

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

$$P(A_i|B) = \frac{P(B|A_i)P(A_i)}{P(B)} = \frac{P(B|A_i)P(A_i)}{\sum_{v_j} P(A_j, B)} = \frac{P(B|A_i)P(A_i)}{\sum_{v_j} P(B|A_j)P(A_j)}$$

- 입력 변수 $x_1, \dots, x_p$ 는 C_j 와 조건부로 서로 독립이라고 가정하여 모델을 간소화 했습니다.

$$P(C_j|x_1, \dots, x_p) = \frac{P(x_1, \dots, x_p|C_j)P(C_j)}{P(x_1, \dots, x_p)}$$

$$P(C_j|x_1, \dots, x_p) \propto P(x_1, \dots, x_p|C_j)P(C_j)$$

Chain Rule에 의해,

$$P(x_1, \dots, x_p|C_j)P(C_j) = P(x_1|x_2, \dots, x_p, C_j)P(x_2|x_3, \dots, x_p, C_j) \dots P(x_p|C_j)P(C_j)$$

입력변수가 C_j 의 조건하에서 서로 독립이라는 가정합니다.

$$P(x_1, \dots, x_p|C_j)P(C_j) = P(x_1|C_j)P(x_2|C_j) \dots P(x_p|C_j)P(C_j)$$

$$= \prod_{i=1}^p (P(x_i|C_j)) P(C_j)$$

$$A, B \text{가 } C \text{에 대하여 조건부 독립이면, } P(A|B, C) = P(A|C)$$

이로써, 학습 과정은

학습데이터에서 각 클래스에 대한 입력 변수들의 조건부확률과 사전확률(Prior)를 구하는 것입니다.

입력 변수들과 클래스의 조건부 확률: $P(x_1|C_j), P(x_2|C_j), \dots, P(x_p|C_j)$

사전 확률(Prior): $P(C_j)$

In [5]: `Image('191.png')`

Out[5]: **[Ex.1]**

df_spam에서 메세지에서 대소문자 구별하지 않고(소문자 처리) 단어를 추출합니다.

단어의 출현 빈도가 5 이상인 것만 남기고,

출현하는 단어를 $w_1, \dots, w_n$ 이라고 할 때, (n은 출현 단어의 수)

$P(w_i \text{ 등장} | target = ham), P(w_i \text{ 등장} | target = spam)$ 을 구합니다.

$$P(w_i \text{ 등장} | target = ham) = \frac{\text{단어 } w_i \text{가 나오는 ham 문서의 수}}{\text{ham인 문서의 수}},$$

$$P(w_i \text{ 등장} | target = spam) = \frac{\text{단어 } w_i \text{가 나오는 spam 문서의 수}}{\text{spam인 문서의 수}}$$

미출현 단어에 의해 확률이 0이 되지 않도록 spam과 ham에서 모든 단어의 빈도에 0.1씩 더 해줍니다.

이와 함께, $P(target = ham), P(target = spam)$ 을 구해 봅니다.

$$P(target = ham) = \frac{\text{ham인 문서의 수}}{\text{전체 문서의 수}}$$

$$P(target = spam) = \frac{\text{spam인 문서의 수}}{\text{전체 문서의 수}}$$

In [4]:

```
'''
코드는 `df_spam` 데이터프레임의 'message' 열에서
스팸 메시지의 단어를 추출하고, 소문자로 변환하며,
중복되지 않는 단어 목록을 생성한
다음 이를 하나의 시리즈로 변환합니다.
```

1. 소문자로 변환:

- `df\_spam` 데이터프레임의 'message' 열의 모든 문자열을 소문자로 변환합니다.

- 예: "Hello World!" -> "hello world!"
- 2. 정규 표현식을 사용하여 단어 추출:
 - 각 메시지 문자열에서 정규 표현식을 사용하여 1자 이상의 소문자로 이루어진 모든 단어를 찾아 리스트로 반환합니다.
 - ``findall('[a-z]{1,})``는 소문자 알파벳으로 이루어진 단어를 모두 찾습니다.
 - 예: "hello world!" -> ``['hello', 'world']``
- 3. 중복되지 않는 단어 목록 생성:
 - ``apply`` 메서드를 사용하여 각 리스트에서 중복된 단어를 제거하고 유일한 단어 목록을 생성합니다.
 - ``np.unique(x)``는 리스트 ``x``에서 중복된 단어를 제거하고 유일한 단어 목록을 반환합니다.
 - 예: ``['hello', 'world', 'hello']`` -> ``['hello', 'world']``
- 4. 리스트를 개별 단어로 분할:
 - ``explode`` 메서드는 각 리스트를 개별 요소로 분할하여 시리즈의 각 행에 하나의 단어만 있도록 합니다.
 - 예: ``['hello', 'world']`` -> ``hello``, ``world``

요약

- ``df_spam['message'].str.lower()``:
'message' 열의 모든 문자열을 소문자로 변환합니다.
- ``str.findall('[a-z]{1,})``:
소문자로 이루어진 단어를 모두 찾아 리스트로 반환합니다.
- ``apply(lambda x : list(np.unique(x)))``:
각 리스트에서 중복된 단어를 제거하고 유일한 단어 목록을 생성합니다.
- ``explode()``:
각 리스트를 개별 요소로 분할하여 시리즈의 각 행에 하나의 단어만 있도록 합니다.

결과적으로 ``s_spam_words``는 ``df_spam``의 모든 메시지에서 중복되지 않는 단어들의 시리즈가 됩니다.
...

message Series를

1. `str.lower()`: 소문자로 바꿉니다.

2. `a-z`가 1회 이상 나오는 문자열들을 추출해냅니다,

```
# 3. 동일 SMS에서 중복된 출현한 단어들은 한 번씩 남도록 합니다.
# 4. explode(): list의 내용들을 풀어서 하나의 Series로 전개합니다.
s_spam_words = df_spam['message'].str.lower()\
               .str.findall('[a-z]{1,}')\
               .apply(lambda x : list(np.unique(x)))\
               .explode()

s_spam_words
```

```
Out[4]: 0      amore
        0    available
        0      buffet
        0      bugis
        0       cine
        ...
5571    its
5571    name
5571    rofl
5571     to
5571    true
Name: message, Length: 79161, dtype: object
```

```
In [5]: # 단어 빈도를 구합니다.
s_word_cnt = s_spam_words.value_counts()
s_word_cnt
```

```
Out[5]: i      2088
        to      1687
        you     1591
        a       1190
        the     1037
        ...
        upon      1
        aspects   1
        rajas     1
        sariyag   1
        someone   1
Name: message, Length: 7785, dtype: int64
```

```
In [6]: '''
이 코드는 이전에 생성된 `s_spam_words` 시리즈에서
특정 조건을 만족하는 단어들만 필터링하여 새로운
```

시리즈로 반환합니다. 조건은 `s\_word\_cnt` 시리즈에서 해당 단어의 출현 빈도가 5 이상인 경우입니다.

1. 출현 빈도 조건 설정:

- `s\_word\_cnt` 시리즈에서 값이 5 이상인 인덱스(단어)만을 선택합니다.
- `s\_word\_cnt`는 각 단어의 출현 빈도를 나타내는 시리즈로 가정합니다.
- 결과는 출현 빈도가 5 이상인 단어들의 시리즈입니다.

2. 해당 단어들의 인덱스 추출:

- 위에서 조건을 만족하는 단어들의 인덱스를 추출합니다.
- 결과는 출현 빈도가 5 이상인 단어들의 인덱스 (단어 목록)입니다.

3. 단어 필터링 조건 설정:

- `s\_spam\_words` 시리즈의 각 단어가 위에서 추출한 단어 목록에 포함되는지 여부를 확인합니다.
- `isin` 메서드는 각 요소가 주어진 목록에 있는지 여부를 불리언 시리즈로 반환합니다.
- 결과는 `True` 또는 `False` 값을 가지는 시리즈로, 각 값은 해당 단어가 조건을 만족하는지 나타냅니다.

4. 조건을 만족하는어들만 선택:

- `loc` 인덱서를 사용하여 조건을 만족하는어들만 선택합니다.
- 결과는 출현 빈도가 5 이상인어들만 포함하는 `s\_spam\_words` 시리즈입니다.

요약

이 코드는 스팸 메시지에서 출현 빈도가 5 이상인어들만을 추출하여 새로운 시리즈를 생성합니다. 이를 통해 흔히 나타나는 중요한어들만을 필터링하여 분석할 수 있습니다.

출현 빈도가 5개 이상인어들만 선택합니다.

```
s_spam_words
    = s_spam_words.loc[s_spam_words\
                        .isin(s_word_cnt\
                            .loc[s_word_cnt >= 5].index)]
s_spam_words
```



```
Out[6]: 0      available
        0      bugis
        0      cine
        0      crazy
        0      e
        ...
        5570     week
        5571     its
        5571     name
        5571     to
        5571     true
Name: message, Length: 69855, dtype: object
```

```
In [7]: # 대상 변수의 빈도수를 구합니다.
s_target_cnt = df_spam['target'].value_counts()
s_target_cnt
```

```
Out[7]: ham      4825
        spam      747
Name: target, dtype: int64
```

```
In [8]: # 대상 변수의 출현 비율을 구합니다. (Prior)
s_target_prob = df_spam['target'].value_counts(normalize=True)
s_target_prob
```

```
Out[8]: ham      0.865937
        spam      0.134063
Name: target, dtype: float64
```

```
In [6]: Image('194.png')
```

Out[6]: 라플라스 스무딩(Laplace Smoothing)은 데이터에서 일부 사건이 일어나지 않은 경우 발생할 수 있는 문제를 해결하기 위해 사용되는 기법입니다. 특히, 확률 추정 문제에서 매우 유용합니다. 나이브 베이즈 분류기와 같은 모델에서 자주 사용됩니다.

문제점

예를 들어, 텍스트 분류에서 특정 단어가 학습 데이터의 어떤 클래스에서도 나타나지 않는 경우, 그 단어가 포함된 문서가 해당 클래스에 속할 확률이 0이 되어 버립니다. 이런 상황은 특히 작은 데이터셋에서 자주 발생합니다. 이런 경우 새로운 데이터에 대해 잘못된 확률 추정이 이루어질 수 있습니다.

라플라스 스무딩의 원리

라플라스 스무딩은 모든 가능한 사건(특징/단어)에 대해 최소한 한 번은 발생한 것으로 가정함으로써 위의 문제를 해결합니다. 이를 위해 각 사건의 빈도수에 1을 더해줍니다.

공식

라플라스 스무딩이 적용된 확률 $P(x_i|y)$ 는 다음과 같이 계산됩니다:

$$P(x_i|y) = \frac{N_{x_i} + 1}{N_y + |V|}$$

여기서:

- N_{x_i} 는 클래스 y 에서 특징 x_i 의 발생 빈도수입니다.

- N_y 는 클래스 y 의 총 단어 수(특징 수)입니다.
- $|V|$ 는 전체 단어 집합(특징 집합)의 크기입니다.

In [7]: `Image('195.png')`

Out[7]: **예시**

다음은 간단한 예시입니다:

데이터셋

- 클래스 A: "apple apple orange"
- 클래스 B: "banana banana banana"

라플라스 스무딩을 적용하지 않은 경우

- $P(apple|A) = \frac{2}{3}$
- $P(orange|A) = \frac{1}{3}$
- $P(banana|A) = 0$
- $P(banana|B) = 1$

라플라스 스무딩을 적용한 경우

- $P(apple|A) = \frac{2+1}{3+3} = \frac{3}{6} = 0.5$
- $P(orange|A) = \frac{1+1}{3+3} = \frac{2}{6} = \frac{1}{3}$
- $P(banana|A) = \frac{0+1}{3+3} = \frac{1}{6}$
- $P(banana|B) = \frac{3+1}{3+3} = \frac{4}{6} = \frac{2}{3}$

위와 같이 라플라스 스무딩을 적용하면 모든 단어에 대해 0이 아닌 확률을 할당할 수 있게 됩니다.

In [8]: `Image('196.png')`

Out[8]: **나이브 베이즈 분류기에서의 적용**

앞서 제시한 코드에서 `MultinomialNB`의 `alpha` 파라미터는 라플라스 스무딩의 정도를 조절하는 역할을 합니다. `alpha` 값이 1이면 라플라스 스무딩을 적용하는 것이고, 이 값을 조정하여 더 많은 스무딩(>1)이나 더 적은 스무딩(<1)을 적용할 수 있습니다. `alpha=0`이면 스무딩이 적용되지 않습니다.

python

 코드 복사

```
clf_nb = MultinomialNB(alpha=1)
```

위 코드에서는 `alpha=1`로 설정하여 라플라스 스무딩을 적용한 다항 나이브 베이즈 분류기를 만들고 있습니다. 이를 통해 모델이 훈련 데이터에서 관찰되지 않은 단어에 대해서도 0이 아닌 확률을 할당할 수 있게 됩니다.

In [9]:

```
'''
이 코드는 스팸 메시지 데이터프레임에서 단어의
출현 확률을 계산합니다. 이를 위해 각 단어가
스팸 또는 정상 메시지에서 나타나는 빈도를 구하고,
라플라스 스무딩(Laplace smoothing)을 적용하여
확률을 계산합니다.

1. 단어 시리즈를 데이터프레임으로 변환:
   - `s_spam_words` 시리즈를 데이터프레임으로 변환합니다.
   - 결과: 단일 열 `message`를 가진 데이터프레임.
2. `target` 컬럼에 따라 그룹화:
   - `s_spam_words`의 인덱스를 사용하여 `df_spam` 데이터프레임의
'''
```

```

    `target` 열에 따라 단어를 그룹화합니다.
    - `target` 열은 메시지가 스팸인지 정상인지 나타내는
      레이블을 포함합니다.
    - 결과: 각 `target` 값(예: 스팸과 정상)에 따라
      그룹화된 데이터프레임.
3. 단어 빈도 계산:
    - 각 그룹 내에서 `message` 열의 단어
      빈도를 계산합니다.
    - `apply(lambda x: x.value_counts())`는 각 그룹에서
      단어의 출현 빈도를 계산합니다.
    - 결과: 각 단어의 출현 빈도를 나타내는
      시리즈가 생성됩니다.
4. 데이터프레임 변환 및 결측값 처리:
    - `unstack` 메서드를 사용하여 데이터를
      피벗 테이블 형태로 변환합니다.
    - `fillna(0)`를 사용하여 결측값을 0으로 채웁니다.
    - `T`를 사용하여 전치(transpose)합니다.
    - 결과: 단어를 행으로, `target` 값을 열로
      가지는 데이터프레임.
5. 라플라스 스무딩 적용:
    - 각 빈도에 0.1을 더하여 라플라스 스무딩을 적용합니다.
    - 이는 확률 계산에서 0값을 방지하고
      모든 단어가 최소한의 확률을 가지도록 합니다.
6. 출현 확률 계산:
    - `s_target_cnt`는 각 `target` 값에 해당하는
      메시지의 수를 나타내는 시리즈입니다.
    - `s_target_cnt + 0.1`를 분모로 사용하여
      확률을 계산합니다.
    - 결과: 각 단어가 스팸과 정상 메시지에서 나타날
      확률을 나타내는 데이터프레임.

### 요약
- `s_spam_words.to_frame()` :
  단어 시리즈를 데이터프레임으로 변환.
- `groupby(df_spam.loc[s_spam_words.index, 'target'])` :
  `target` 컬럼에 따라 단어를 그룹화.
- `apply(lambda x: x.value_counts())` :
  각 그룹 내 단어 빈도 계산.
- `unstack().fillna(0).T` :
  데이터를 피벗 테이블 형태로 변환하고
  결측값을 0으로 채운 후 전치.

```

- ``+ 0.1``: 라플라스 스무딩 적용.
- ``/ (s_target_cnt + 0.1)``: 단어의 출현 확률 계산.

결과는 단어가 스팸과 정상 메시지에서 나타날 확률을 나타내는 데이터프레임입니다.

'''

$P(w_i|target=ham)$, $P(w_i|target=spam)$ 의
확률을 구합니다.

ham과 spam별 단어 출현빈도를 구합니다.
빈도수가 없는 단어가 나오면 확률을 0으로 하면,
대상 변수를 분류하는데 유력한 단어가 나온다고 하더라도,
빈도수가 없는 단어가 섞여 있으면
해당 클래스일 확률이 0이 됩니다.
이러한 문제를 완화하기 위해 빈도수가 없더라도
최소빈도수(0.1)를 넣어 줍니다.
전체합으로 나누어 ham, spam에 대한
조건부 확률로 만들어줍니다.
이 때, 모든 단어 빈도에 더한 최소빈도수(0.1) 만큼을
df_target_cnt에 더하여 조건부 확률의
최대값이 1이 되도록합니다.

"""

아래와 동일 루틴입니다.

```
df_word_prob = (
    pd.crosstab(index=[df_spam.loc[s_spam_words.
        index, 'target']],
        columns=s_spam_words) + 0.1
).divide(s_target_cnt + 0.1, axis=0).T"""
```

```
df_word_prob = (s_spam_words.to_frame().groupby(
    df_spam.loc[s_spam_words.index, 'target']
)['message'].apply(lambda x: x.value_counts())\
    .unstack().fillna(0).T + 0.1) \
    / (s_target_cnt + 0.1)
```

```
df_word_prob.head()
```

```
Out[9]:
```

	target	ham	spam
a	0.183851	0.405702	
aathi	0.001264	0.000134	
abiola	0.002300	0.000134	
able	0.005409	0.000134	
about	0.029865	0.009503	

```
In [3]: Image('192.png')
```

```
Out[3]: [Ex.2]
```

단어의 출현 유무를 통해 spam/ham을 분류하는 나이브 베이즈를 이용하여 구하고자 합니다.

x_i 를 SMS에 w_i 가 등장하는 여부를 C_j 를 SMS의 target이 ham/spam인 여부를 나타냅니다.

$\prod_{i=1}^p (P(x_i|C_j)) P(C_j)$ 곱 연산은 곱의 수가 많아질 수록 floating point 표현 범위를 벗어날 가능성이 높습니다.

단조 증가 함수인 log를 사용을 하면 별다른 처리 없이 대소 관계를 비교할 수 있고,

곱을 합으로 바꿀 수 있어, floating point 연산에 대한 안정성을 가져올 수 있습니다.

다음과 같이 log를 적용하여 아래의 판별함수 값을 target_ham, target_spam인 로그 확률을 만들어 봅니다.

$$\log(P(C_j|x_1, \dots, x_p)) \propto \log(\prod_{i=1}^p (P(x_i|C_j)) P(C_j)) = \sum_{i=1}^p \log(P(x_i|C_j)) + \log(P(C_j))$$

이를 기반으로 spam/ham 을 분류합니다.

```
In [10]: '''
코드는 스팸 메시지에서 단어의 로그 확률을 계산하고,
이를 이용해 각 메시지가 스팸일 확률을 추정합니다.
```


1. 로그 확률 계산:
 - `df_word_prob`는 이전 단계에서 계산한 각 단어의 출현 확률을 담고 있는 데이터프레임입니다.
 - `np.log(df_word_prob)`를 통해 각 단어의 확률에 로그를 취합니다. 로그 변환은 확률 곱셈을 더하기로 변환하여 계산의 안정성과 효율성을 높입니다.
 - 결과는 각 단어의 로그 확률을 담고 있는 데이터프레임입니다.
2. 단어 로그 확률 매핑:
 - `s_spam_words.map(df_word_prob_log['spam'])`: 각 단어를 `df_word_prob_log` 데이터프레임의 'spam' 열에 매핑하여 각 단어의 로그 확률을 얻습니다.
 - `.dropna()`: 로그 확률이 없는 단어(즉, 스팸 확률이 0이었던 단어)를 제거합니다.
 - `.reset_index()`: 인덱스를 리셋하여 그룹화에 적합한 형태로 변환합니다.
 - `.groupby('index')['message'].sum()`: 원래 메시지 인덱스별로 그룹화하여 각 메시지의 단어 로그 확률을 합산합니다. 즉, 각 메시지에 포함된 단어들의 로그 확률을 모두 더합니다.
 - `+ np.log(s_target_prob.loc['spam'])`: 스팸일 확률의 로그 값을 더합니다. 이는 베이즈 정리를 적용하기 위한 사전 확률(prior probability)입니다.
 - 결과는 각 메시지가 스팸일 로그 확률을 나타내는 시리즈입니다.

요약

- `np.log(df_word_prob)`: 단어의 확률에 로그를 취하여 계산의 안정성과 효율성을 높입니다.
- `s_spam_words.map(df_word_prob_log['spam'])`: 각 단어의 로그 확률을 매핑합니다.
- `dropna().reset_index().groupby('index')['message'].sum()`: 메시지 인덱스별로 그룹화하여 각 메시지의 단어 로그 확률을 합산합니다.
- `+ np.log(s_target_prob.loc['spam'])`: 스팸일 확률의 로그 값을 더하여 최종 로그 확률을 계산합니다.

결과는 각 메시지가 스팸일 로그 확률을 나타내는 시리즈입니다.

이를 통해 메시지가 스팸일 가능성을 추정할 수 있습니다.

'''

```
# 문서별로  $P(x_i/spam)P(spam)$ 
# = 등장한 단어에서 spam일 로그 확률을 구합니다.
# 1. 문서에 등장한 단어를 spam 문서에 등장할
# 로그 확률로 바꿉니다.
# 2. 문서별로 확률의 합을 구합니다.
# 3. spam일 확률을 더합니다.
df_word_prob_log = np.log(df_word_prob)
s_spam_logprob = s_spam_words.map(df_word_prob_log['spam'])\
    .dropna().reset_index()\
    .groupby('index')['message'].sum() + \
    np.log(s_target_prob.loc['spam'])
s_spam_logprob
```

```
Out[10]: index
0      -90.732008
1      -35.419485
2      -70.843435
3      -44.802511
4      -69.673543
...
5567   -65.807471
5568   -28.744702
5569   -34.698459
5570  -112.719810
5571   -15.994286
Name: message, Length: 5560, dtype: float64
```

```
In [11]: '''
코드는 이전에 계산한 스팸 메시지에 대한 로그 확률과
유사하게, 각 메시지가 정상(ham) 메시지일
확률을 추정합니다.

1. 로그 확률 계산:
- 이 단계는 이전에 계산한 `df_word_prob_log`
  데이터프레임을 사용합니다.
  이는 각 단어의 출현 확률에 로그를 취한 값입니다.
2. 단어 로그 확률 매핑:
- `s_spam_words` 시리즈의 각 단어를
  `df_word_prob_log` 데이터프레임의 'ham' 열에
  매핑하여 각 단어의 로그 확률을 얻습니다.
3. 결측값 제거 및 인덱스 리셋:
```

- 로그 확률이 없는 단어(즉, 정상 메시지에서 나타나지 않은 단어)를 제거합니다.
 - ``reset_index()``를 사용하여 인덱스를 리셋하고, 그룹화에 적합한 형태로 변환합니다.
4. 메시지별 단어 로그 확률 합산:
- 원래 메시지 인덱스별로 그룹화하여 각 메시지의 단어 로그 확률을 합산합니다. 즉, 각 메시지에 포함된 단어들의 로그 확률을 모두 더합니다.
5. 정상 메시지 사전 확률 로그값 더하기:
- 정상(ham) 메시지일 확률의 로그 값을 더합니다. 이는 베이즈 정리를 적용하기 위한 사전 확률(prior probability)입니다.

전체 과정 요약

이 코드는 각 메시지가 정상(ham) 메시지일 로그 확률을 계산합니다.

1. 로그 확률을 계산한 데이터프레임 사용:
``df_word_prob_log``는 각 단어가 스팸과 정상 메시지에서 나타날 로그 확률을 포함합니다.
2. 단어 로그 확률 매핑:
``s_spam_words`` 시리즈의 각 단어를 ``df_word_prob_log['ham']``에 매핑하여 각 단어의 정상 메시지 로그 확률을 얻습니다.
3. 결측값 제거 및 인덱스 리셋:
로그 확률이 없는 단어를 제거하고, 인덱스를 리셋하여 그룹화에 적합한 형태로 변환합니다.
4. 메시지별 단어 로그 확률 합산:
각 메시지의 단어 로그 확률을 합산하여 각 메시지에 대한 총 로그 확률을 계산합니다.
5. 정상 메시지 사전 확률 로그값 더하기:
정상 메시지일 사전 확률의 로그 값을 더하여 최종 로그 확률을 계산합니다.

요약

- ``np.log(df_word_prob)``:
단어의 확률에 로그를 취하여 계산의 안정성과 효율성을 높입니다.
- ``s_spam_words.map(df_word_prob_log['ham'])``:
각 단어의 로그 확률을 매핑합니다.
- ``dropna().reset_index().groupby('index')['message'].sum()``:

```

    메시지 인덱스별로 그룹화하여
    각 메시지의 단어 로그 확률을 합산합니다.
- `+ np.log(s_target_prob.loc['ham'])`:
    정상(ham) 메시지일 확률의 로그 값을 더하여
    최종 로그 확률을 계산합니다.

결과는 각 메시지가 정상(ham)일 로그 확률을 나타내는 시리즈입니다.
이를 통해 메시지가 정상일 가능성을 추정할 수 있습니다.
'''

```

```

# 문서별로  $P(x_i|spam)P(ham)$ 
# = 등장한 단어에서 ham일 로그 확률을 구합니다.
# 1. 문서에 등장한 단어를 ham 문서에 등장할
# 로그 확률로 바꿉니다.
# 2. 문서별로 확률의 합을 구합니다.
# 3. spam일 확률을 더합니다.
s_ham_logprob = s_spam_words.map(df_word_prob_log['ham'])\
    .dropna().reset_index()\
    .groupby('index')['message'].sum() + \
    np.log(s_target_prob.loc['ham'])
s_ham_logprob

```

```

Out[11]: index
0      -78.296232
1      -21.688316
2     -132.949220
3      -29.549207
4      -46.526959
...
5567   -128.425125
5568    -21.004267
5569    -27.172805
5570    -87.335977
5571    -15.006446
Name: message, Length: 5560, dtype: float64

```

```

In [12]: '''
    코드는 스팸 필터링 모델의 예측 결과를 `df_spam`
    데이터프레임에 추가하고, 실제 값과 예측 값이
    다른 오분류 데이터를 출력하는 과정을 포함합니다.
'''

```

1. 로그 확률 비교 및 예측 생성:
 - ``s_spam_logprob``와 ``s_ham_logprob``를 비교합니다.
스팸 로그 확률이 정상(ham) 로그 확률보다 큰 경우
'spam'으로 예측하고, 그렇지 않은 경우
'ham'으로 예측합니다.
 - ``np.where`` 함수를 사용하여 이 비교의 결과를
'spam' 또는 'ham'으로 반환합니다.
 - ``pd.Series``를 사용하여 이 결과를 시리즈로 변환하고,
인덱스를 ``s_spam_logprob``의 인덱스로 설정합니다.
2. 예측 결과를 데이터프레임에 추가:
 - ``assign`` 메서드를 사용하여 원래 ``df_spam``
데이터프레임에 ``target_prd`` 열을 추가합니다.
이 열은 예측된 목표 변수(target)를 포함합니다.
 - ``fillna('ham')`` 메서드를 사용하여 결측값을
'ham'으로 채웁니다. 이는 예측값이 결측일 경우,
해당 메시지를 정상 메시지로 간주합니다.
3. 오분류 데이터 출력:
 - ``loc`` 인덱서를 사용하여 실제 목표 변수(``target``)와
예측된 목표 변수(``target_prd``)가
다른 행을 선택합니다.
 - 이 결과로 오분류된 메시지들만 포함된
데이터프레임을 출력합니다.

요약

- 로그 확률 비교:
각 메시지에 대해 스팸 로그 확률과
정상 로그 확률을 비교하여 예측 결과를 생성합니다.
- 예측 결과 추가:
``df_spam`` 데이터프레임에 예측된
목표 변수(``target_prd``)를 추가합니다.
- 오분류 데이터 출력:
실제 목표 변수(``target``)와 예측된
목표 변수(``target_prd``)가 다른 행들을 출력합니다.

이를 통해 모델의 성능을 평가하고,
어떤 메시지들이 잘못 분류되었는지 확인할 수 있습니다.
'''

spam일 확률과 ham일 확률을 비교하여
클래스 예측을 합니다.

```
# 5회 이상 출현하지 않아, 단어기반 확률을 구할 수 없는
# 경우는 빈도가 높은 'ham'으로 예측합니다.
df_spam_result = df_spam.assign(\
    target_prd \
    = pd.Series(np.where(s_spam_logprob > s_ham_logprob, \
                        'spam', 'ham'), \
                index=s_spam_logprob.index)\
).fillna('ham')

# 오분류 데이터를 출력합니다.
df_spam_result.loc[
    df_spam_result['target'] != df_spam_result['target_prd']
]
```

Out[12]:

	target	message	target_prd
45	ham	No calls..messages..missed calls	spam
68	spam	Did you hear about the new "Divorce Barbie"? I...	ham
69	ham	I plane to give on this month end.	spam
74	ham	U can call me now...	spam
180	ham	Hi! You just spoke to MANEESHA V. We'd like to...	spam
...	...	...	...
5157	ham	K k:) sms chat with me.	spam
5202	ham	WOT STUDENT DISCOUNT CAN U GET ON BOOKS?	spam
5315	ham	Hahaha..use your brain dear	spam
5475	ham	Dhoni have luck to win some big title.so we wi...	spam
5553	ham	Hahaha..use your brain dear	spam

130 rows × 3 columns

In [13]: '''
이 코드는 스팸 필터링 모델의 정확도를 계산합니다.

```

1. 실제 값과 예측 값 비교:
- `df_spam_result` 데이터프레임의 `target` 열(실제 값)과
  `target_prd` 열(예측 값)을 비교합니다.
- 각 행에 대해 실제 값과 예측 값이 같으면 `True`,
  다르면 `False`를 반환하는 불리언 시리즈를 생성합니다.

2. 평균 계산:
- `np.mean` 함수는 입력된 불리언 시리즈에서
  `True`의 비율을 계산합니다.
- 불리언 시리즈에서 `True`는 1, `False`는 0으로 간주되므로,
  `np.mean` 함수는 정확도를 계산합니다.

### 요약
- 비교 연산:
  `df_spam_result['target'] == df_spam_result['target_prd']`는
  실제 값과 예측 값이 일치하는지 여부를 나타내는
  불리언 시리즈를 만듭니다.
- 정확도 계산: `np.mean`을 사용하여 이 불리언 시리즈의
  평균을 계산하면, 모델의 예측이 실제 값과 일치한 비율,
  즉 정확도를 구할 수 있습니다.

결과적으로 이 코드는 모델의 예측이 실제 값과
얼마나 잘 맞는지를 평가하는 정확도를 계산합니다.
'''

# 예측 정확도입니다.
np.mean(df_spam_result['target'] == df_spam_result['target_prd'])

```

Out[13]: 0.9766690595836325

In [5]: Image('193.png')

Out[5]: **sklearn.naive_bayes.MultinomialNB**

입력 변수가 명목형 데이터에서 범주의 특징을 나타내는 이산형(discrete) 변수일 때 사용합니다.

위의 예제에서는 모든 입력 변수들이 문장에 등장하는 단어의 빈도수를 의미합니다.

주요 하이퍼 파라미터

이름	설명
alpha	가법 평활화(Additive Smoothing) 평활 계수입니다.

- 가법 평활화

$$\hat{\theta}_{ci} = \frac{N_{ci} + \alpha}{N_c + \alpha n}$$

α 가 작을 수록 평활화가 적게 적용 되어 학습 데이터를 더 반영하고, 클수록 평활화가 크게 적용 되어 학습 데이터를 덜 반영하게 됩니다.

$N_{ci} = \sum_{i=1}^c x_i$ 클래스 c에서 i 속성이 등장한 수입니다.

$N_c = \sum_{i=1}^n N_{ci}$ 클래스 c에서 등장한 속성의 총합입니다.

[Ex.3]

df_spam에서 80%를 학습 데이터 df_train으로

나머지 20%는 평가 데이터는 df_test로 나누어 봅니다.

최소한 5건의 메시지에 등장한 단어의 빈도를 입력 변수(속성)으로

사용하여 target 변수를 예측하는 나이브 베이즈 모델을 만듭니다.

단어의 빈도 측정은 df_train을 대상으로 합니다.

이 때, 가법 평활 계수는 1로 합니다.

성능 측정 기준은 `f1_score` 입니다. `Positive`는 `spam` 클래스 입니다.

In [14]: `from sklearn.model_selection import train_test_split`

```
df_train, df_test
= train_test_split(df_spam, train_size=0.8,
                  random_state=123)
```

In [15]:

```
'''
코드는 텍스트 데이터에 대해 나이브 베이즈
분류기를 사용하여 스팸 메시지를 예측하고,
테스트 데이터와 훈련 데이터에 대한
F1 점수를 계산합니다.

1. 파이프라인 생성:
- `make_pipeline` 함수는 여러 단계를 연속적으로
  처리하는 파이프라인을 만듭니다.
- 첫 번째 단계는 `CountVectorizer`입니다.
  이 단계는 텍스트 데이터를 숫자 벡터로 변환합니다.
- `lowercase=True`:
  모든 텍스트를 소문자로 변환합니다.
- `min_df=5`:
  최소 5개 이상의 문서에 나타나는 단어만
  벡터에 포함시킵니다.
- 두 번째 단계는 `MultinomialNB`입니다.
  이 단계는 다항 나이브 베이즈 분류기를 사용하여
  분류를 수행합니다.
- `alpha=1`:
  라플라스 스무딩 파라미터입니다.
  1로 설정하여 모든 단어의 확률이
  0이 되지 않도록 합니다.

2. 모델 훈련:
- `fit` 메서드를 사용하여
  훈련 데이터(`df_train['message']`)와
  목표 변수(`df_train['target']`)를
  바탕으로 모델을 학습시킵니다.
- 훈련 데이터의 텍스트 메시지를 벡터화한 후,
  나이브 베이즈 분류기를 사용하여 학습합니다.
'''
```

3. F1 점수 계산:

- `f1_score` 함수는 모델의 F1 점수를 계산합니다.
F1 점수는 모델의 정밀도(Precision)와 재현율(Recall)의 조화 평균을 나타냅니다.
- `clf_nb.predict(df_test['message'])`:
테스트 데이터의 텍스트 메시지에 대한 예측을 수행합니다.
- `clf_nb.predict(df_train['message'])`:
훈련 데이터의 텍스트 메시지에 대한 예측을 수행합니다.
- `pos_label='spam'`:
스팸 메시지를 양성 클래스로 간주하여 F1 점수를 계산합니다.
- 두 개의 F1 점수를 계산하여 튜플 형태로 반환합니다.
첫 번째는 테스트 데이터에 대한 F1 점수이고,
두 번째는 훈련 데이터에 대한 F1 점수입니다.

요약

- 파이프라인 생성: 텍스트 벡터화 및
나이브 베이즈 분류기를 포함하는 파이프라인을 만듭니다.
- 모델 훈련: 훈련 데이터를 사용하여 모델을 학습시킵니다.
- F1 점수 계산: 테스트 데이터와 훈련 데이터에 대해
모델의 F1 점수를 계산하고, 이를 튜플 형태로 반환합니다.

결과적으로 이 코드는 훈련된 나이브 베이즈 분류기의 성능을
테스트 데이터와 훈련 데이터에서 평가합니다.

F1 점수는 모델의 성능을 종합적으로 평가하는 지표로,
정밀도와 재현율을 모두 고려합니다.

'''

```
from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.pipeline import make_pipeline
from sklearn.metrics import f1_score

clf_nb = make_pipeline(
    CountVectorizer(lowercase=True, min_df=5),
    MultinomialNB(alpha=1)
)
clf_nb.fit(df_train['message'], df_train['target'])
(
    f1_score(df_test['target'],
            clf_nb.predict(df_test['message']),
```

```

        pos_label='spam'),
    f1_score(df_train['target'],
            clf_nb.predict(df_train['message']),
            pos_label='spam')
)

```

Out[15]: (0.9442622950819671, 0.957841483979764)

[Ex.4]

df_spam에서 80%를 학습 데이터 df_train으로
나머지 20%는 평가 데이터는 df_test로 나누어 봅니다.

최소한 5건의 메시지에 등장한 단어의 빈도를
입력 변수(속성)으로 사용하여 target 변수를
예측하는 나이브 베이즈 모델을 만듭니다.

단어의 빈도 측정은 df_train을 대상으로 합니다.

가법 평활 계수가 아래 리스트에 따라 변할 때,
학습 데이터와 평가 데이터의 성능 비교를 해봅니다.

성능 측정 기준은 AUC(Area Under the receiver
characteristic Curve)입니다.

Positive는 spam 클래스 입니다.

```
alphas = [0.5, 0.8, 1.0, 1.2, 1.5]
```

```

In [16]: '''
코드는 여러 가지 `alpha` 값에 대해 나이브 베이즈 분류기를
학습시키고, 훈련 데이터와 테스트 데이터에 대한

```

AUC (Area Under the Curve) 점수를 계산하여 이를 그래프로 시각화합니다.

1. 알파 값 설정:

- 라플라스 스무딩 파라미터 ``alpha``의 여러 값을 리스트로 정의합니다.

2. AUC 점수를 저장할 리스트 초기화:

- 훈련 데이터와 테스트 데이터에 대한 AUC 점수를 저장할 빈 리스트를 초기화합니다.

3. 각 알파 값에 대한 모델 학습 및 AUC 계산:

- 각 ``alpha`` 값에 대해 파이프라인을 생성합니다.
``CountVectorizer``는 텍스트를 벡터화하고,
``MultinomialNB``는 다항 나이브 베이즈 분류기를 사용합니다.
- ``fit`` 메서드를 사용하여 모델을 훈련시킵니다.
- ``predict_proba`` 메서드를 사용하여 각 클래스에 대한 확률을 예측하고, 양성 클래스(스팸)의 확률을 가져옵니다.
- ``roc_auc_score`` 함수를 사용하여 훈련 데이터와 테스트 데이터에 대한 AUC 점수를 계산하고, 각각 ``auc_train``과 ``auc_test`` 리스트에 추가합니다.

4. AUC 점수 시각화:

- 그래프를 그리기 위한 새로운 **Figure**를 생성합니다.
- ``alphas`` 값을 x축으로, 훈련 데이터의 AUC 점수를 y축으로 하여 선 그래프를 그립니다.
- ``alphas`` 값을 x축으로, 테스트 데이터의 AUC 점수를 y축으로 하여 또 다른 선 그래프를 그립니다.
- x축과 y축 라벨을 설정하고, 범례를 추가합니다.

요약

- 여러 ``alpha`` 값에 대한 모델 학습: 각 ``alpha`` 값에 대해 나이브 베이즈 분류기를 학습시킵니다.
- AUC 점수 계산: 훈련 데이터와 테스트 데이터에 대해 AUC 점수를 계산합니다.
- 그래프로 시각화: ``alpha`` 값에 따른 훈련 데이터와 테스트 데이터의 AUC 점수를 그래프로 시각화합니다.

결과적으로 이 코드는 다양한 ``alpha`` 값에 대해 나이브 베이즈 모델의 성능을 평가하고, 이를 통해 최적의 ``alpha`` 값을 선택하는 데 도움을 줍니다.

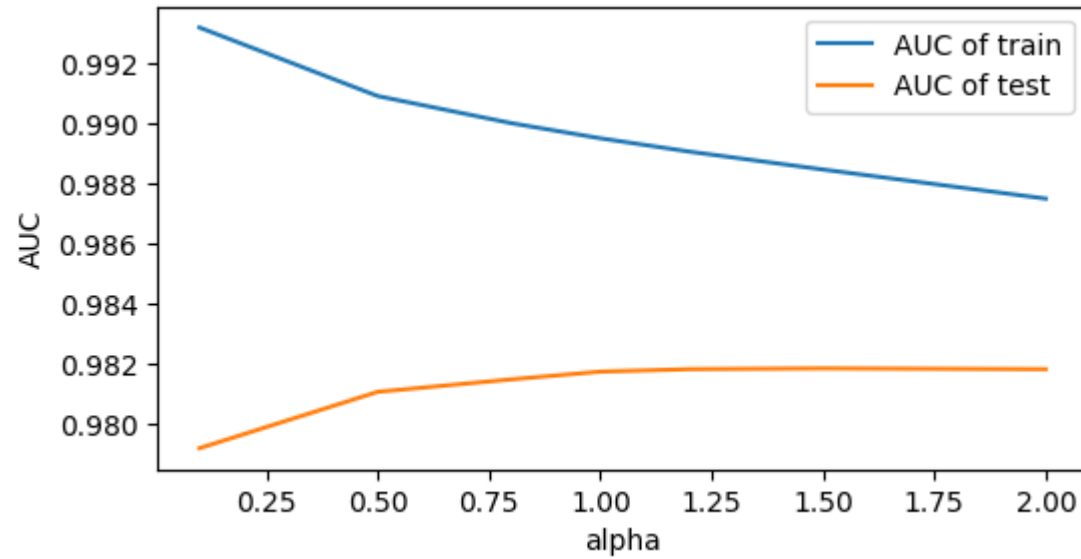
AUC 점수는 모델의 분류 성능을 평가하는 중요한 지표입니다.

...

```
from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.pipeline import make_pipeline
from sklearn.metrics import roc_auc_score

alphas = [0.1, 0.5, 0.8, 1.0, 1.2, 1.5, 2.0]
auc_train, auc_test = list(), list()
for alpha in alphas:
    clf_nb = make_pipeline(
        CountVectorizer(lowercase=True, min_df=5),
        MultinomialNB(alpha=alpha)
    )
    clf_nb.fit(df_train['message'], df_train['target'])
    auc_test.append(
        roc_auc_score(df_test['target'], \
            clf_nb.predict_proba(df_test['message']))[:, 1])
    )
    auc_train.append(
        roc_auc_score(df_train['target'], \
            clf_nb.predict_proba(df_train['message']))[:, 1])
    )

plt.figure(figsize=(6, 3))
plt.plot(alphas, auc_train, label='AUC of train')
plt.plot(alphas, auc_test, label='AUC of test')
plt.gca().set_xlabel('alpha')
plt.gca().set_ylabel('AUC')
plt.legend()
plt.show()
```



In [9]: Image('197.png')

Out[9]: **나이브 베이즈의 장단점**

장점

- 간단하고 학습이 빠릅니다.
- 상대적으로 적은 양의 훈련 데이터로도 효과성이 높습니다.
- 동일한 척도의 여러 속성을 지닌 데이터(ex. 텍스트 분류, 추천...)에 강점이 있습니다.

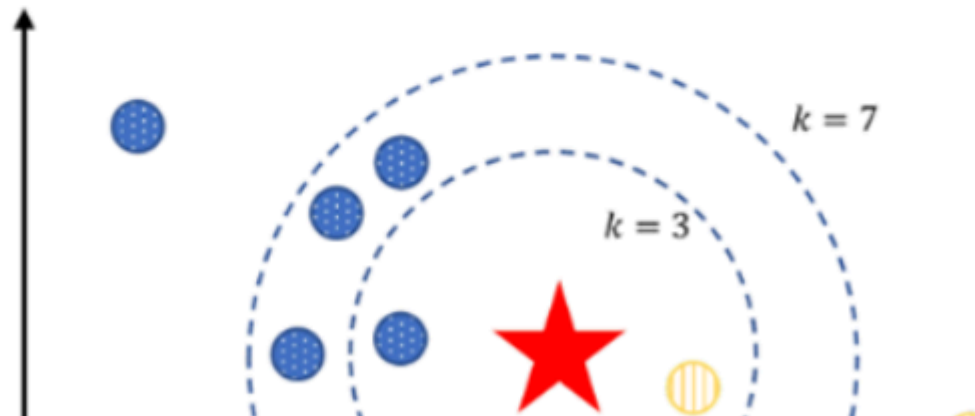
단점

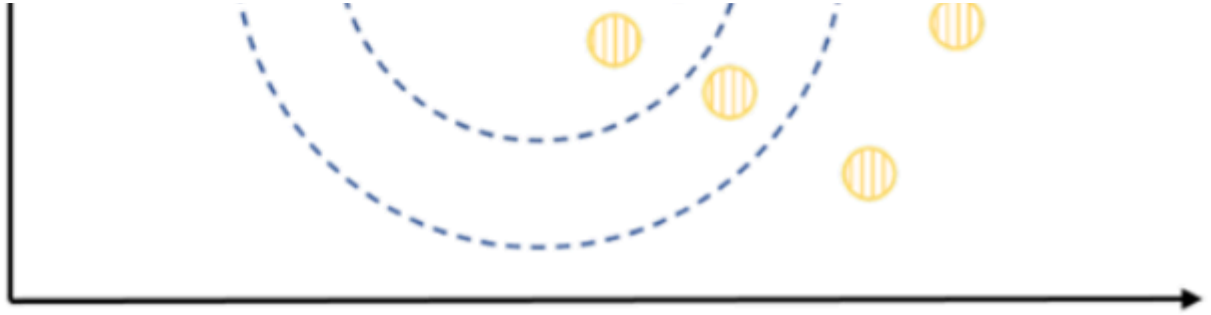
- 조건부 독립이라는 가정이 실제로는 부적절한 경우가 많습니다.

2. K 최근접 이웃 모델

K 최근접 이웃은 학습 데이터를 저장해두고, 입력 데이터에서 근접한 K개의 데이터를 뽑아 예측을 합니다.

최근접 이웃의 수 K와 근접도를 나타내는 거리 측정 방법이 주요 구성 요소입니다.





In [10]: `Image('198.png')`

Out[10]: **특징**

최근접 이웃의 수를 늘릴 수록 편향은 증가하고 분산을 감소합니다.

분류

K 개의 근접 이웃에 해당하는 학습 데이터에서 다수의 클래스를 예측값으로 합니다.

클래스의 수가 동일할 경우에는 전체셋에 대해 다수의 셋을 사용합니다.

회귀

K 개의 근접 이웃에 해당하는 대상 변수의 평균을 예측값으로 합니다.

sklearn.neighbors.KNeighborsClassifier, KNeighborsRegressor

주요 하이퍼 파라미터

이름	설명
n_neighbors	근접 이웃 수(K)
metric	거리 측정 방법

[Ex.3]

위 예제에서 모델을 K-최근접 이웃모델로 바꾸고
근접이웃의 수를 아래 리스트에 따라 바꿀 때,
df_train과 df_test의 성능 변화를 관찰해봅니다.

성능 측정 지표는 Accuracy입니다.

```
n_neighbors_list = [1, 3, 5, 10, 20, 50, 100]
```

In [17]:

```
'''
코드는 다양한 k값에 대해 K-최근접 이웃(KNN) 분류기의
성능을 평가하고, 학습 데이터와 테스트 데이터에서의
정확도(Accuracy) 점수를 시각화합니다.
여기서 k는 KNN 분류기에서 고려하는 이웃의 수를 나타냅니다.

1. 텍스트 데이터를 벡터화:
- `CountVectorizer`를 사용하여 텍스트 데이터를
  벡터화합니다. `lowercase=True`는 텍스트를 소문자로
  변환하고, `min_df=5`는 최소 빈도가 5 이상인
  단어만을 벡터화합니다.
- `fit_transform` 메서드를 사용하여
  학습 데이터(`df_train['message']`)를 벡터화하고,
  변환된 데이터를 `X_train`에 저장합니다.
- `transform` 메서드를 사용하여
  테스트 데이터(`df_test['message']`)를 벡터화하고,
  변환된 데이터를 `X_test`에 저장합니다.

2. 리스트 초기화:
- 다양한 k 값들을 담은 리스트
  `n_neighbors_list`를 정의합니다.
- 학습 데이터와 테스트 데이터에서의 정확도 점수를
  저장할 리스트 `acc_train`과 `acc_test`를 초기화합니다.

3. k 값에 따른 모델 학습 및 평가:
- 각 k 값에 대해 루프를 돌면서 다음 작업을 수행합니다:
  - `KNeighborsClassifier`를 생성합니다.
    `n_neighbors=i`는 이웃의 수를 i로 설정하고,
    `metric='euclidean'`은 유클리드 거리를
    사용하도록 설정합니다.
  - 학습 데이터를 이용해 KNN 분류기를 학습시킵니다.
  - 테스트 데이터에 대한 정확도 점수를 계산하여
    `acc_test` 리스트에 추가합니다.
  - 학습 데이터에 대한 정확도 점수를 계산하여 `
    acc_train` 리스트에 추가합니다.
'''
```

4. 결과 시각화:

- k 값에 따른 학습 데이터와 테스트 데이터의 정확도 점수를 각각 플롯합니다.
- x축 레이블을 'n_neighbors', y축 레이블을 'Accuracy'로 설정합니다.
- 범례를 추가하여 학습 데이터와 테스트 데이터의 정확도 점수를 구분합니다.

요약

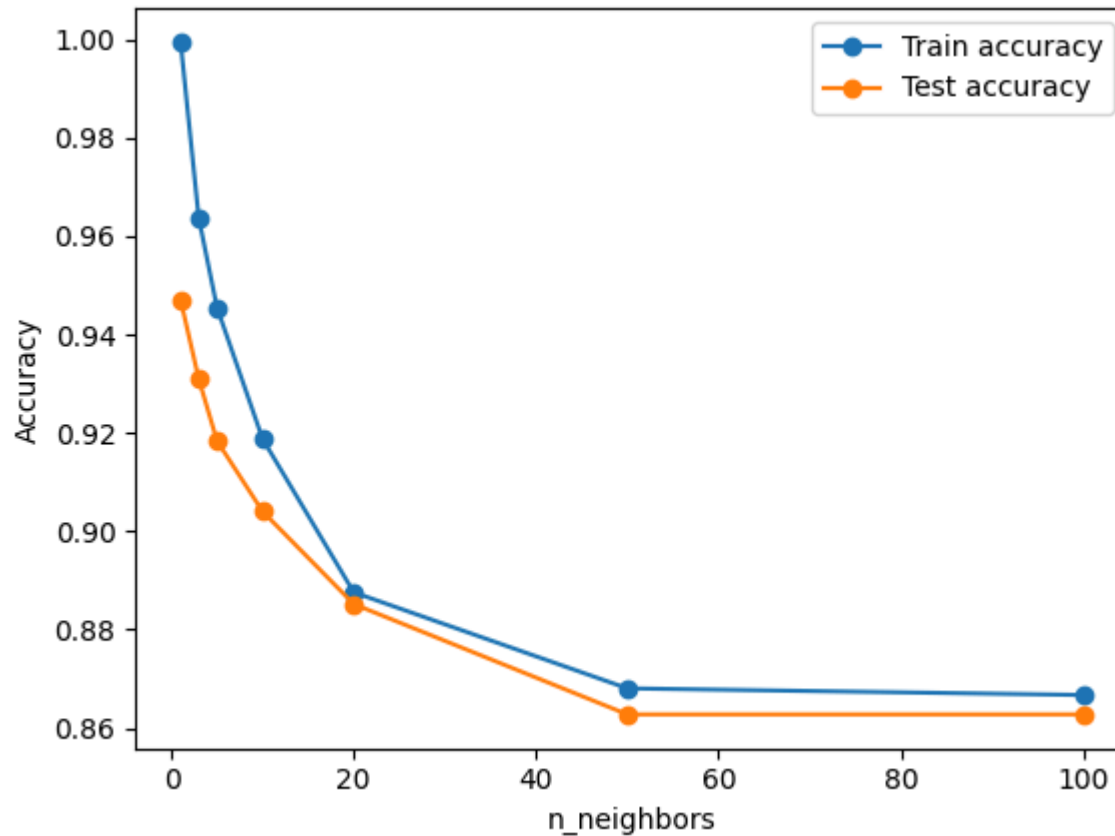
이 코드는 다양한 k 값에 대해 KNN 모델의 성능을 학습 데이터와 테스트 데이터에서 평가하고, 그 결과를 시각화하여 k 이 모델 성능에 미치는 영향을 분석합니다. 정확도(Accuracy) 점수를 사용하여 분류기의 성능을 평가하며, 이를 통해 최적의 k 값을 선택할 수 있습니다.

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
```

```
cvec = CountVectorizer(lowercase=True, min_df=5)
X_train = cvec.fit_transform(df_train['message'])
X_test = cvec.transform(df_test['message'])
n_neighbors_list = [1, 3, 5, 10, 20, 50, 100]
acc_train, acc_test = list(), list()
```

```
for i in n_neighbors_list:
    clf_knn = KNeighborsClassifier(
        n_neighbors=i,
        metric='euclidean'
    )
    clf_knn.fit(X_train, df_train['target'])
    acc_test.append(
        accuracy_score(df_test['target'],
                        clf_knn.predict(X_test))
    )
    acc_train.append(
        accuracy_score(df_train['target'],
                        clf_knn.predict(X_train))
    )
```

```
In [18]: plt.plot(n_neighbors_list, acc_train,
                  label='Train accuracy', marker='o')
plt.plot(n_neighbors_list, acc_test,
          label='Test accuracy', marker='o')
plt.gca().set_xlabel('n_neighbors')
plt.gca().set_ylabel('Accuracy')
plt.legend()
plt.show()
```



K-최근접 이웃의 장단점

장점

- 간단하고 이해하기 쉽습니다.
- 데이터 탑재 외에는 학습 과정이 필요하지 않아

빠르고 간단하게 모델을 완성할 수 있습니다.

단점

- 데이터의 수가 많아 질수록 예측시 계산 비용이 높아집니다.
- 입력 변수(속성)의 수가 많아질수록 성능 저하의 여지가 크고,

이를 대응하기 위한 학습 데이터의 필요량이 속성이 많아질 수록 커집니다.

- 데이터가 특정 공간에 몰려 있거나, 드문드문 배치되었을 때,

지엽적으로 성능의 편차가 발생합니다.

In []: